

```

FSController.java
package com.centricsoftware.core.controller;
import com.alibaba.fastjson.JSONArray;
import com.centricsoftware.core.service.FSService;
import com.centricsoftware.commons.dto.ResEntity;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

/**
 * 飞书 Aily 接口调用控制层
 * 接口路径: /feishu/aily
 */
@Slf4j
@RestController
@RequestMapping("/feishu") // 父路径 feishu
public class FSController {

    @Autowired
    private FSService fsService;

    /**
     * 调用飞书 Aily 接口
     * @param jsonArray 包含 PLM 前端传入的 URL 信息
     * @return 响应内容
     * @throws Exception
     */
    @RequestMapping(value = "/aily/gb2760", method = RequestMethod.POST, produces =
"application/json; charset=utf-8")
    public ResEntity callGB2760(@RequestBody JSONArray jsonArray) throws Exception {
        log.info("飞书 Aily 接口接收参数: {}", jsonArray.toJSONString());
        ResEntity result = fsService.callAily(jsonArray); // 调用 FSService 中的方法
        return result;
    }

    // 其他可能的接口可以按相同方式添加，例如：
    // @RequestMapping(value = "/aily/otherEndpoint", method = RequestMethod.POST)
    // public ResEntity callOtherEndpoint(@RequestBody JSONArray jsonArray) { ... }
}

```

```

FSService.java
package com.centricsoftware.core.service;

import com.alibaba.fastjson.JSONArray;

```

```
import com.alibaba.fastjson.JSONObject;
import com.centricsoftware.commons.dto.ResEntity;
import com.centricsoftware.commons.dto.WebResponse;
import com.centricsoftware.commons.em.ResCode;
import com.centricsoftware.commons.utils.NodeUtil;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.lang3.StringEscapeUtils;
import org.springframework.stereotype.Service;
```

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
```

```
@Service
```

```
@Slf4j
```

```
public class FSService {
```

```
    // 固定的 app_id 和 app_secret
```

```
    private static final String APP_ID = "cli_a83c1dce493e900e";
```

```
    private static final String APP_SECRET = "XI3Ab3u68POC4sLvzLrrVhhmetycsJWV";
```

```
    private static final String SERVER_URL = "https://open.feishu.cn/open-apis";
```

```
    /**
```

```
    * 主要逻辑: 提取 PLM 参数 → 获取 token → 构造 query → 调用 Aily → 返回结果
```

```
    */
```

```
    public ResEntity callAily(JSONArray jsonArray) throws Exception {
```

```
        try {
```

```
            for (int i = 0; i < jsonArray.size(); i++) {
```

```
                JSONObject jsonObject = jsonArray.getJSONObject(i);
```

```
                String url = jsonObject.getString("url");
```

```
                log.info("开始处理 URL: {}", url);
```

```
                // step1: 获取 RecipeDataSheet 对象信息
```

```
                Map<String, String> recipeMap = NodeUtil.JSqueryAttributes(url);
```

```
                String revisionUrl = recipeMap.get("CurrentRevision");
```

```
                log.info("CurrentRevision = {}", revisionUrl);
```

```

// step2: 进入 RecipeDataSheetRevision 获取 C8_RDS_Category
Map<String, String> revisionMap = NodeUtil.JSqueryAttributes(revisionUrl);
String categoryCode = revisionMap.get("C8_RDS_Category");
log.info("category_code = {}", categoryCode);

// step3: 从 RecipeDataSheet 获取 RecipeFoodLabel 字段
String foodLabelUrl = recipeMap.get("RecipeFoodLabel");
log.info("RecipeFoodLabel = {}", foodLabelUrl);

// step4: 进入 FoodLabelDataSheet → CurrentRevision →
FoodLabelDataSheetRevision
Map<String, String> foodLabelMap =
NodeUtil.JSqueryAttributes(foodLabelUrl);
String foodRevisionUrl = foodLabelMap.get("CurrentRevision");
Map<String, String> foodRevisionMap =
NodeUtil.JSqueryAttributes(foodRevisionUrl);

// step5: 获取 Ingredients (refvector 数组)
String ingredientsRef = foodRevisionMap.get("Ingredients");
log.info("Ingredients refvector = {}", ingredientsRef);

List<JSONObject> ingredientsList = new ArrayList<>(); // 修改为
List<JSONObject>
if (ingredientsRef != null && !ingredientsRef.isEmpty()) {
    String[] ingredientUrls = ingredientsRef.replace("[", "").
        .replace("]", "")
        .split(",");
    for (String ingUrl : ingredientUrls) {
        ingUrl = ingUrl.trim();
        if (ingUrl.isEmpty()) continue;
        Map<String, String> ingMap = NodeUtil.JSqueryAttributes(ingUrl);
        String name = ingMap.get("$Name");
        log.info("name:" + name);
        String ratio = ingMap.get("CalculatedPct");
        String formattedRatio = (ratio != null) ? String.format("%.0f%%",
Double.parseDouble(ratio) * 100) : "0%"; // 格式化比例为百分比
        JSONObject ingJson = new JSONObject();
        ingJson.put("name", name); // 确保 name 被添加到 JSON 对
象中
        ingJson.put("ratio", formattedRatio);
        ingredientsList.add(ingJson); // 直接添加 JSON 对象
    }
}

```

```

// step6: 组织 query JSON（压缩为一行）
JSONObject queryJson = new JSONObject();
queryJson.put("category_code", categoryCode);
queryJson.put("ingredients", ingredientsList); // 使用 List<JSONObject>
而不是转换为字符串
String queryStr = queryJson.toJSONString();
log.info("query 参数: {}", queryStr);

// step7: 获取飞书 token
String token = getFeishuToken();
log.info("获取到的飞书 token: {}", token);

// step8: 调用飞书 Aily 接口
JSONObject globalVariable = new JSONObject();
globalVariable.put("query", queryStr);
JSONObject bodyJson = new JSONObject();
bodyJson.put("global_variable", globalVariable);

// 发送请求
String responseBody = sendHttpRequest(token, bodyJson);
// 打印原始响应内容，防止乱码（尝试重新以 UTF-8 解码）
try {
    String decodedResponse = new
String(responseBody.getBytes(StandardCharsets.ISO_8859_1), StandardCharsets.UTF_8);
    log.info("Aily 接口原始响应（UTF-8）: {}", decodedResponse);
} catch (Exception e) {
    log.warn("Aily 接口响应打印异常，原始内容: {}", responseBody);
}

// ** 修改 8**：直接将 Aily 接口返回的 `output` 字段写入
FoodLabelDataSheetRevision 的 Testresults 字段
JSONObject respJson = JSONObject.parseObject(responseBody);
String output = respJson.getJSONObject("data").getString("output");
log.info("Aily 接口返回的 output: {}", output); // ** 修改 9**：打印
`output` 字段，确认其内容

String cleanOutput = output
    .replaceFirst("\\{\\\"answer\\\":\\\"\", \"\") // 去掉开头的 {\"answer\": \"
    .replaceAll(\"\\\"\\\"\\\"\", \"\") // 去掉结尾的 \"
    .replace(\"{\", \"\") // 去掉花括号 {
    .replace(\"}\", \"\") // 去掉花括号 }
    .replaceAll(\"\\n+\", \"\") // 去掉多余的换行符

```

```

        .replaceAll("&lt;", "<")                // 将转义的<还原
        .replaceAll("&gt;", ">")                // 将转义的>还原
        .replaceAll("&quot;", "\"");            // 将转义的"还原

String escapedOutput = StringEscapeUtils.escapeXml10(cleanOutput);
log.info("转义后的: ",escapedOutput);
// **替换换行符为 HTML <br> 或保留原有的换行符**
cleanOutput = escapedOutput.replace("\n", "<br>");
log.info("替换换行符后的: ",cleanOutput);

// 使用 OAServiceUtils.setFiledVale 方法直接更新 Testresults 字段
OAServiceUtils.setFiledVale(foodRevisionUrl, "Testresults",
cleanOutput,"string");

return WebResponse.success(ResCode.SUCCESS); // 直接返回成功, 使用默
认的消息和代码
    }
} catch (Exception e) {
    log.error("调用飞书 Aily 接口出错: ", e);
    return WebResponse.error("调用飞书 Aily 接口出错: " + e.getMessage());
}
return WebResponse.error("FAIL");
}

/**
 * 获取飞书 tenant_access_token
 */
private String getFeishuToken() throws Exception {
    String tokenUrl = SERVER_URL + "/auth/v3/tenant_access_token/internal";
    HttpURLConnection connection = (HttpURLConnection) new
URL(tokenUrl).openConnection();
    connection.setRequestMethod("POST");
    connection.setDoOutput(true);
    connection.setRequestProperty("Content-Type", "application/json");

    // 创建请求体
    String requestBody = String.format("{\"app_id\":\"%s\",\"app_secret\":\"%s\"}",
APP_ID, APP_SECRET);
    try (OutputStream os = connection.getOutputStream()) {
        byte[] input = requestBody.getBytes(StandardCharsets.UTF_8);
        os.write(input, 0, input.length);
    }
}

```

```

        // 获取响应
        BufferedReader reader = new BufferedReader(new
InputStreamReader(connection.getInputStream()));
        StringBuilder response = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            response.append(line);
        }
        reader.close();

        // 解析返回的 JSON
        String json = response.toString();
        log.info("获取 token 响应: {}", json);
        JSONObject jsonResponse = JSONObject.parseObject(json);
        return jsonResponse.getString("tenant_access_token");
    }

    /**
     * 发送 HTTP 请求到飞书 Aily 接口
     */
    private String sendHttpRequest(String token, JSONObject bodyJson) throws Exception {
        String url =
"https://open.feishu.cn/open-apis/aily/v1/apps/spring_24208e1b94__c/skills/skill_1cdf9d94735
d/start";
        HttpURLConnection connection = (HttpURLConnection) new
URL(url).openConnection();
        // 设置超时时间（单位：毫秒）
        connection.setConnectTimeout(50000); // 设置连接超时时间为 30 秒
        connection.setReadTimeout(50000); // 设置读取超时时间为 30 秒
        connection.setRequestMethod("POST");
        connection.setDoOutput(true);
        connection.setRequestProperty("Authorization", "Bearer " + token);
        connection.setRequestProperty("Content-Type", "application/json");

        // 创建请求体
        String requestBody = bodyJson.toJSONString();
        try (OutputStream os = connection.getOutputStream()) {
            byte[] input = requestBody.getBytes(StandardCharsets.UTF_8);
            os.write(input, 0, input.length);
        }

        // 获取响应
        BufferedReader reader = new BufferedReader(new

```

```

InputStreamReader(connection.getInputStream());
    StringBuilder response = new StringBuilder();
    String line;
    while ((line = reader.readLine()) != null) {
        response.append(line);
    }
    reader.close();

    return response.toString();
}
}

```

//XML 代码块

```

<View Id="IngredientListGlobal" xsi:type="Property" UseDefaultCard="false">
    <Toolbar>
        <Action                                Id="UpdateGlobalIngredientList"
PostAction="postActionRedrawNode" Control="Button">
            <Rule Exp="attr('State') == 'Revision.State:DRAFT'"/>
            <Request                                Module="FormulatedProduct"
Operation="FoodLabelUpdateGlobalIngredientList"/>
            </Action>
            <Action Id="Ingredienttesting" OnAction="Site.Ingredienttesting" >
                <Rule Exp="currentRoleName()=='Admin' || currentRoleName()=='应用
研发工程师'"/>
            </Action>
        </Toolbar>
        <Heading                                Index="Global"                                Id="IngredientPerLanguage"
Name="GlobalIngredientList"/>
        <!-- <Actions>
            <Action                                Id="FlatIngredientToggle"
ControllerClass="FormulatedProduct.FoodLabel.FlatIngredientsToggleController"
ActionClass="FormulatedProduct.FlatIngredientsToggleButton"
PostAction="csi.page.refreshViews"/>
        </Actions> -->
    </View>

```

//JS 代码块

```

Ingredienttesting: async function (ac) {
    console.log("ac:", ac);
    const writer = ac.m_writer;
    const mcache = ac.m_cache;

    try {
        // 找到当前节点
    }
}

```

```

const currentNode = mcache.getNode(ac.m_urlNode);
// 找到上级节点
const parentNode = mcache.getNode(currentNode.__Parent__);
// 找到上上级节点 (RecipeDataSheet)
const grandParentNode = mcache.getNode(parentNode.__Parent__);

// 找到 ParentDataSheetRevision 对象
const parentDataSheetRevUrl = currentNode.ParentDataSheetRevision;
const parentDataSheetRevObj = mcache.getNode(parentDataSheetRevUrl);

// 判断 C8_RDS_Category 是否为空
if (!parentDataSheetRevObj.C8_RDS_Category ||
parentDataSheetRevObj.C8_RDS_Category.trim() == "" ||
parentDataSheetRevObj.C8_RDS_Category == "C8_category:") {
    jp.alert("请先到汇总页面选择 GB2760 分类");
    return;
}

// 新增：确认弹窗
const confirmStr = "点击该按钮后，将会进行配料合规检测，是否继续？";
layer.confirm(confirmStr, {
    btn: ['确定', '取消'],
    closeBtn: 0
}, async function (index) {
    // 用户点击“确定”，关闭确认框
    layer.close(index);

    // 构造请求体
    const payload = [
        { url: grandParentNode.$URL }
    ];
    console.log("发送到后端的请求体：", JSON.stringify(payload, null, 2));

    // ☒ 使用我们新建的 POST 方法调用后端
    const result = await requestSRMServiceApiPost('feishu/aily/gb2760',
payload);

    console.log("后端返回结果：", result);

    // ☒ 结果处理逻辑保持一致
    if (!result.success) {
        jp.alert("检测失败，原因： " + (result.data || '未知错误'));
        return;
    }
}

```



```

        writer.execute({ sync: true });
        jp.alert("检测成功！");
        location.reload();
    }, function (index) {
        // 用户点击“取消”
        layer.close(index);
        console.log("用户取消了配料合规检测");
    });

    } catch (err) {
        console.error("请求出错：", err);
        jp.alert("检测请求异常，请稍后重试！");
    }

    return true;
},

```

```

data_import.py
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
GB 2760 数据导入脚本
将 final_gb2760_data.xlsx 中的数据导入到 MySQL 数据库
"""

```

```

import pandas as pd
import pymysql
import logging
from typing import Dict, List, Any
import sys
import os
from datetime import datetime

# 配置日志
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('data_import.log', encoding='utf-8'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)

```

```

class GB2760DataImporter:
    """GB 2760 数据导入器"""

    def __init__(self, db_config: Dict[str, Any]):
        """初始化数据库配置"""
        self.db_config = db_config
        self.connection = None
        self.excel_file = 'final_gb2760_data.xlsx'

        # 统计信息
        self.import_stats = {
            'additive': 0,
            'additive_usage': 0,
            'food_category': 0,
            'category_additive': 0,
            'processing_aid': 0,
            'enzyme': 0,
            'flavor': 0
        }

    def connect_database(self) -> bool:
        """连接数据库"""
        try:
            self.connection = pymysql.connect(**self.db_config)
            logger.info("☑ 数据库连接成功")
            return True
        except Exception as e:
            logger.error(f"✗ 数据库连接失败: {e}")
            return False

    def close_database(self):
        """关闭数据库连接"""
        if self.connection:
            self.connection.close()
            logger.info("🔒 数据库连接已关闭")

    def check_and_setup_database(self) -> bool:
        """检查和设置数据库，清空表数据"""
        try:
            # 不指定数据库的连接
            temp_config = self.db_config.copy()
            if 'database' in temp_config:
                del temp_config['database']

```

```

temp_connection = pymysql.connect(**temp_config)
cursor = temp_connection.cursor()

# 检查数据库是否存在
cursor.execute("SHOW DATABASES LIKE 'gb2760_db'")
db_exists = cursor.fetchone() is not None

if not db_exists:
    logger.info("❌ 数据库 gb2760_db 不存在，创建数据库...")
    cursor.execute("CREATE DATABASE gb2760_db CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci")
    logger.info("✅ 数据库创建成功")
else:
    logger.info("❌ 数据库 gb2760_db 已存在")

# 使用数据库
cursor.execute("USE gb2760_db")

# 检查表是否存在
cursor.execute("SHOW TABLES")
tables = [table[0] for table in cursor.fetchall()]

if tables:
    logger.info(f"❌ 发现 {len(tables)} 个表，清空表数据...")

    # 关闭外键检查
    cursor.execute("SET FOREIGN_KEY_CHECKS = 0")

    # 清空所有表数据（保留表结构）
    for table in tables:
        try:
            cursor.execute(f"TRUNCATE TABLE {table}")
            logger.info(f"✅ 清空表数据: {table}")
        except Exception as e:
            # 如果是视图，则删除
            try:
                cursor.execute(f"DROP VIEW IF EXISTS {table}")
                logger.info(f"✅ 删除视图: {table}")
            except Exception as e2:
                logger.warning(f"❌ 处理表失败 {table}: {e2}")

    # 重新启用外键检查
    cursor.execute("SET FOREIGN_KEY_CHECKS = 1")

```

```

        logger.info("☑ 表数据清空完成")
    else:
        logger.info("☑ 数据库为空，无需清理")

    temp_connection.commit()
    cursor.close()
    temp_connection.close()

    # 重新连接到指定数据库
    self.db_config['database'] = 'gb2760_db'
    if self.connection:
        self.connection.close()
    self.connection = pymysql.connect(**self.db_config)

    return True

except Exception as e:
    logger.error(f"❌ 数据库设置失败: {e}")
    return False

def execute_sql_file(self, sql_file: str) -> bool:
    """执行 SQL 文件"""
    try:
        with open(sql_file, 'r', encoding='utf-8') as f:
            sql_content = f.read()

        # 分割 SQL 语句（简单分割，忽略注释中的分号）
        sql_statements = []
        current_statement = ""
        in_comment = False

        for line in sql_content.split('\n'):
            line = line.strip()
            if not line:
                continue

            if line.startswith('--'):
                continue

            current_statement += line + " "

            if line.endswith(';'):
                if current_statement.strip():
                    sql_statements.append(current_statement.strip())
                    current_statement = ""
    
```

```

        sql_statements.append(current_statement.strip())
        current_statement = ""

    cursor = self.connection.cursor()

    for sql in sql_statements:
        if sql.strip() and not sql.strip().startswith('--'):
            try:
                cursor.execute(sql)
            except Exception as e:
                logger.warning(f"SQL 执行警告: {str(e)[:100]}...")
                continue

    self.connection.commit()
    cursor.close()
    logger.info(f"☑ SQL 文件执行完成: {sql_file}")
    return True

except Exception as e:
    logger.error(f"✗ SQL 文件执行失败: {e}")
    return False

def import_additives(self) -> bool:
    """导入添加剂基本信息"""
    try:
        logger.info("📁 开始导入添加剂基本信息...")

        df = pd.read_excel(self.excel_file, sheet_name='添加剂基本信息')
        logger.info(f"读取到 {len(df)} 条添加剂基本信息")

        cursor = self.connection.cursor()

        sql = """
        INSERT INTO additive (
            id, name_zh, name_en, cns_code, ins_code,
            function_category, detail_url, quality_standard,
            jecfa_spec, remarks
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        """

        success_count = 0
        for _, row in df.iterrows():
            try:
                values = (

```

```

        row['id'],
        row['name_zh'],
        row['name_en'] if pd.notna(row['name_en']) else None,
        row['cns_code'] if pd.notna(row['cns_code']) else None,
        row['ins_code'] if pd.notna(row['ins_code']) else None,
        row['function_category'] if pd.notna(row['function_category']) else
None,

        row['detail_url'] if pd.notna(row['detail_url']) else None,
        row['quality_standard'] if pd.notna(row['quality_standard']) else
None,

        row['jecfa_spec'] if pd.notna(row['jecfa_spec']) else None,
        row['remarks'] if pd.notna(row['remarks']) else None
    )

    cursor.execute(sql, values)
    success_count += 1

except Exception as e:
    logger.warning(f"导入添加剂失败: {row['name_zh']} - {e}")
    continue

self.connection.commit()
cursor.close()

self.import_stats['additive'] = success_count
logger.info(f"☒ 添加剂基本信息导入完成: {success_count}/{len(df)}")
return True

except Exception as e:
    logger.error(f"☒ 导入添加剂基本信息失败: {e}")
    return False

def import_additive_usage(self) -> bool:
    """导入添加剂使用限量"""
    try:
        logger.info("☒ 开始导入添加剂使用限量...")

        df = pd.read_excel(self.excel_file, sheet_name='添加剂使用限量')
        logger.info(f"读取到 {len(df)} 条添加剂使用限量记录")

        cursor = self.connection.cursor()

        sql = """
INSERT INTO additive_usage (

```

```

        additive_id, additive_name, category_code,
        food_name, max_usage, usage_remarks
    ) VALUES (%s, %s, %s, %s, %s, %s)
    """

    success_count = 0
    batch_size = 1000
    batch_data = []

    for _, row in df.iterrows():
        try:
            values = (
                row['additive_id'],
                row['additive_name'],
                row['category_code'] if pd.notna(row['category_code']) else None,
                row['food_name'] if pd.notna(row['food_name']) else None,
                row['max_usage'] if pd.notna(row['max_usage']) else None,
                row['usage_remarks'] if pd.notna(row['usage_remarks']) else None
            )

            batch_data.append(values)

            if len(batch_data) >= batch_size:
                cursor.executemany(sql, batch_data)
                success_count += len(batch_data)
                batch_data = []

                if success_count % 5000 == 0:
                    logger.info(f"已导入 {success_count} 条使用限量记录")

        except Exception as e:
            logger.warning(f"导入使用限量失败: {row['additive_name']} - {e}")
            continue

    # 导入剩余批次
    if batch_data:
        cursor.executemany(sql, batch_data)
        success_count += len(batch_data)

    self.connection.commit()
    cursor.close()

    self.import_stats['additive_usage'] = success_count
    logger.info(f"☑ 添加剂使用限量导入完成: {success_count}/{len(df)}")

```

```

        return True

    except Exception as e:
        logger.error(f"✘ 导入添加剂使用限量失败: {e}")
        return False

def import_food_categories(self) -> bool:
    """导入食品分类基本信息"""
    try:
        logger.info("🔗 开始导入食品分类基本信息...")

        df = pd.read_excel(self.excel_file, sheet_name='分类基本信息')
        logger.info(f"读取到 {len(df)} 条分类基本信息")

        cursor = self.connection.cursor()

        sql = """
        INSERT INTO food_category (
            id, category_code, category_name,
            detail_url, description, standards
        ) VALUES (%s, %s, %s, %s, %s, %s)
        """

        success_count = 0
        for _, row in df.iterrows():
            try:
                values = (
                    row['id'],
                    row['category_code'],
                    row['category_name'],
                    row['detail_url'] if pd.notna(row['detail_url']) else None,
                    row['description'] if pd.notna(row['description']) else None,
                    row['standards'] if pd.notna(row['standards']) else None
                )

                cursor.execute(sql, values)
                success_count += 1

            except Exception as e:
                logger.warning(f"导入分类失败: {row['category_name']} - {e}")
                continue

        self.connection.commit()
        cursor.close()

```



```

self.import_stats['food_category'] = success_count
logger.info(f"☑ 食品分类基本信息导入完成: {success_count}/{len(df)}")
return True

```

```

except Exception as e:
    logger.error(f"✗ 导入食品分类基本信息失败: {e}")
    return False

```

```

def import_category_additives(self) -> bool:

```

```

    """导入分类-添加剂关联"""

```

```

    try:

```

```

        logger.info("🔄 开始导入分类-添加剂关联...")

```

```

        df = pd.read_excel(self.excel_file, sheet_name='分类-添加剂关联')
        logger.info(f"读取到 {len(df)} 条分类-添加剂关联记录")

```

```

        cursor = self.connection.cursor()

```

```

        sql = """
INSERT INTO category_additive (
    category_id, category_code, category_name, additive_name,
    `function`, max_usage, cns_code, ins_code, remarks
) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        """

```

```

        success_count = 0

```

```

        batch_size = 1000

```

```

        batch_data = []

```

```

        for _, row in df.iterrows():

```

```

            try:

```

```

                values = (

```

```

                    row['category_id'],

```

```

                    row['category_code'],

```

```

                    row['category_name'] if pd.notna(row['category_name']) else

```

None,

```

                    row['additive_name'],

```

```

                    row['function'] if pd.notna(row['function']) else None,

```

```

                    row['max_usage'] if pd.notna(row['max_usage']) else None,

```

```

                    row['cns_code'] if pd.notna(row['cns_code']) else None,

```

```

                    row['ins_code'] if pd.notna(row['ins_code']) else None,

```

```

                    row['remarks'] if pd.notna(row['remarks']) else None

```

```

                )

```

```

        batch_data.append(values)

    if len(batch_data) >= batch_size:
        cursor.executemany(sql, batch_data)
        success_count += len(batch_data)
        batch_data = []

    if success_count % 5000 == 0:
        logger.info(f"已导入 {success_count} 条关联记录")

except Exception as e:
    logger.warning(f"导入关联失败: {row['additive_name']} - {e}")
    continue

# 导入剩余批次
if batch_data:
    cursor.executemany(sql, batch_data)
    success_count += len(batch_data)

self.connection.commit()
cursor.close()

self.import_stats['category_additive'] = success_count
logger.info(f"☑ 分类-添加剂关联导入完成: {success_count}/{len(df)}")
return True

except Exception as e:
    logger.error(f"✗ 导入分类-添加剂关联失败: {e}")
    return False

def import_other_data(self) -> bool:
    """导入其他数据（加工助剂、酶制剂、香精香料）"""

    # 导入加工助剂
    try:
        logger.info("🔄 开始导入加工助剂...")
        df = pd.read_excel(self.excel_file, sheet_name='加工助剂')

        cursor = self.connection.cursor()
        sql = """
        INSERT INTO processing_aid (
            name_zh, name_en, `function`, usage_scope,
            cas_number, remarks, raw_data

```

```

) VALUES (%s, %s, %s, %s, %s, %s, %s)
"""

success_count = 0
for _, row in df.iterrows():
    try:
        values = (
            row['name_zh'],
            row['name_en'] if pd.notna(row['name_en']) else None,
            row['function'] if pd.notna(row['function']) else None,
            row['usage_scope'] if pd.notna(row['usage_scope']) else None,
            row['cas_number'] if pd.notna(row['cas_number']) else None,
            row['remarks'] if pd.notna(row['remarks']) else None,
            row['raw_data'] if pd.notna(row['raw_data']) else None
        )
        cursor.execute(sql, values)
        success_count += 1
    except Exception as e:
        logger.warning(f"导入加工助剂失败: {row['name_zh']} - {e}")

self.connection.commit()
cursor.close()
self.import_stats['processing_aid'] = success_count
logger.info(f"☑ 加工助剂导入完成: {success_count}/{len(df)}")

except Exception as e:
    logger.error(f"✗ 导入加工助剂失败: {e}")

# 导入酶制剂
try:
    logger.info("🔄 开始导入酶制剂...")
    df = pd.read_excel(self.excel_file, sheet_name='酶制剂')

    cursor = self.connection.cursor()
    sql = """
    INSERT INTO enzyme (
        name_zh, name_en, enzyme_source, usage_scope,
        ec_number, cas_number, remarks, raw_data
    ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
    """

    success_count = 0
    for _, row in df.iterrows():
        try:

```

```

        values = (
            row['name_zh'],
            row['name_en'] if pd.notna(row['name_en']) else None,
            row['enzyme_source'] if pd.notna(row['enzyme_source']) else
None,

            row['usage_scope'] if pd.notna(row['usage_scope']) else None,
            row['ec_number'] if pd.notna(row['ec_number']) else None,
            row['cas_number'] if pd.notna(row['cas_number']) else None,
            row['remarks'] if pd.notna(row['remarks']) else None,
            row['raw_data'] if pd.notna(row['raw_data']) else None
        )
        cursor.execute(sql, values)
        success_count += 1
    except Exception as e:
        logger.warning(f"导入酶制剂失败: {row['name_zh']} - {e}")

self.connection.commit()
cursor.close()
self.import_stats['enzyme'] = success_count
logger.info(f"☑ 酶制剂导入完成: {success_count}/{len(df)}")

except Exception as e:
    logger.error(f"✗ 导入酶制剂失败: {e}")

# 导入香精香料
try:
    logger.info("🔗 开始导入香精香料...")
    df = pd.read_excel(self.excel_file, sheet_name='香精香料')

    cursor = self.connection.cursor()
    sql = """
INSERT INTO flavor (
    category, name_zh, name_en, type, code,
    fema_no, classification_table, remarks, raw_data
) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
"""

    success_count = 0
    batch_size = 500
    batch_data = []

    for _, row in df.iterrows():
        try:
            values = (

```

```

        row['category'],
        row['name_zh'],
        row['name_en'] if pd.notna(row['name_en']) else None,
        row['type'] if pd.notna(row['type']) else None,
        row['code'] if pd.notna(row['code']) else None,
        row['fema_no'] if pd.notna(row['fema_no']) else None,
        row['classification_table'] if pd.notna(row['classification_table'])
else None,

        row['remarks'] if pd.notna(row['remarks']) else None,
        row['raw_data'] if pd.notna(row['raw_data']) else None
    )
    batch_data.append(values)

    if len(batch_data) >= batch_size:
        cursor.executemany(sql, batch_data)
        success_count += len(batch_data)
        batch_data = []

except Exception as e:
    logger.warning(f"导入香精香料失败: {row['name_zh']} - {e}")

# 导入剩余批次
if batch_data:
    cursor.executemany(sql, batch_data)
    success_count += len(batch_data)

self.connection.commit()
cursor.close()
self.import_stats['flavor'] = success_count
logger.info(f"☑ 香精香料导入完成: {success_count}/{len(df)}")

except Exception as e:
    logger.error(f"✗ 导入香精香料失败: {e}")

return True

def print_import_summary(self):
    """打印导入总结"""
    print("\n" + "="*60)
    print("📊 GB 2760 数据导入完成！")
    print("="*60)

    total_records = sum(self.import_stats.values())

```

```

        for table_name, count in self.import_stats.items():
            table_display = {
                'additive': '添加剂基本信息',
                'additive_usage': '添加剂使用限量',
                'food_category': '食品分类基本信息',
                'category_additive': '分类-添加剂关联',
                'processing_aid': '加工助剂',
                'enzyme': '酶制剂',
                'flavor': '香精香料'
            }
            print(f"    {table_display[table_name]}: {count:,} 条")

        print(f"\n    总导入记录数: {total_records:,} 条")
        print(f"    导入时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
        print(f"    数据源: final_gb2760_data.xlsx")
        print(f"    目标数据库: {self.db_config.get('database', 'gb2760_db')}")
        print("="*60)

def main():
    """主函数"""

    # 数据库配置（在这里设置您的 MySQL 密码）
    db_config = {
        'host': 'localhost',
        'port': 3306,
        'user': 'root',
        'password': 'la123456', # 在这里填入您的 MySQL 密码，例如: 'your_password'
        'charset': 'utf8mb4',
        'autocommit': False
    }

    print("\n GB 2760-2024 数据导入工具")
    print("="*60)
    print("本工具将 final_gb2760_data.xlsx 中的数据导入到 MySQL 数据库")
    print("\n 密码配置: 如需修改 MySQL 密码，请在脚本中的 db_config['password']处修改")
    print()

    # 检查密码是否已配置
    if not db_config['password']:
        password = input("请输入 MySQL 密码 (直接回车使用空密码): ")
        if password:
            db_config['password'] = password
    else:

```

```

        print(f"🔑 使用预设密码: {'*' * len(db_config['password'])}")

print()

# 创建导入器
importer = GB2760DataImporter(db_config)

# 连接数据库
if not importer.connect_database():
    print("❌ 无法连接数据库，请检查配置")
    return

try:
    # 0. 检查和设置数据库
    print("🔑 第零步：检查和设置数据库...")
    if importer.check_and_setup_database():
        print("✅ 数据库检查和设置成功")
    else:
        print("❌ 数据库设置失败")
        return

    print()

    # 1. 执行数据库结构脚本
    print("🔑 第一步：创建数据库结构...")
    if importer.execute_sql_file('database_schema.sql'):
        print("✅ 数据库结构创建成功")
    else:
        print("❌ 数据库结构创建失败")
        return

    print()

    # 2. 导入数据
    print("🔑 第二步：导入数据...")

    # 按依赖顺序导入
    steps = [
        ("添加剂基本信息", importer.import_additives),
        ("食品分类基本信息", importer.import_food_categories),
        ("添加剂使用限量", importer.import_additive_usage),
        ("分类-添加剂关联", importer.import_category_additives),
        ("其他数据", importer.import_other_data)
    ]

```

```

        for step_name, step_func in steps:
            print(f"\n正在导入: {step_name}")
            if not step_func():
                print(f"{step_name}导入出现问题，但继续执行后续步骤")

# 3. 打印总结
importer.print_import_summary()

finally:
    importer.close_database()

if __name__ == "__main__":
    main()

final_gb2760_scraper.py
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
GB 2760-2024 最终完整数据爬虫
爬取所有真实数据，建立完整的多对多关系
"""

import requests
from bs4 import BeautifulSoup
import pandas as pd
import urllib3
import time
import logging
from typing import List, Dict, Optional
import re

# 禁用 SSL 警告
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

# 配置日志
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler(__import__('pathlib').Path(__file__).parent / 'final_scraper.log',
encoding='utf-8'),
        logging.StreamHandler()
    ]

```



```

)
logger = logging.getLogger(__name__)

def __improt__(param):
    pass

class FinalGB2760Scraper:
    """GB 2760 最终完整数据爬虫"""

    def __init__(self):
        self.base_url = "https://gb2760.cfsa.net.cn"
        self.session = self._create_session()

        # 数据存储
        self.additives_basic = []          # 添加剂基本信息
        self.additives_usage = []          # 添加剂使用限量
        self.categories_basic = []         # 分类基本信息
        self.category_additives = []       # 分类-添加剂关联
        self.food_categories = []          # 食品分类列表
        self.processing_aids = []          # 加工助剂
        self.enzymes = []                  # 酶制剂
        self.spices = []                   # 香精香料

    def _create_session(self):
        """创建请求会话"""
        session = requests.Session()
        session.verify = False
        session.headers.update({
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36',
            'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
            'Accept-Language': 'zh-CN,zh;q=0.9,en;q=0.8',
            'Connection': 'keep-alive',
        })
        return session

# ===== 添加剂数据爬取 =====

def get_additive_list(self) -> List[Dict]:
    """获取添加剂列表"""
    logger.info("开始获取添加剂列表...")

```

```

try:
    url = self.base_url + "/additives.html"
    response = self.session.get(url, timeout=15)
    response.encoding = 'utf-8'
    soup = BeautifulSoup(response.text, 'html.parser')

    additives_list = []
    tables = soup.find_all('table')

    if tables:
        table = tables[0]
        rows = table.find_all('tr')
        logger.info(f"找到 {len(rows)-1} 个添加剂")

        for i, row in enumerate(rows[1:], 1):
            cells = row.find_all(['td', 'th'])
            if len(cells) >= 3:
                name_zh = cells[0].get_text(strip=True)
                name_en = cells[1].get_text(strip=True)
                cns_code = cells[2].get_text(strip=True)
                ins_code = cells[3].get_text(strip=True) if len(cells) > 3 else ""
                function_category = cells[4].get_text(strip=True) if len(cells) > 4 else ""

            link = cells[0].find('a')
            if link:
                detail_url = link.get('href', "")
                if detail_url.startswith('/'):
                    detail_url = self.base_url + detail_url

                additive_id = None
                if '/faid/' in detail_url:
                    try:
                        additive_id =
int(detail_url.split('/faid/')[1].split('.')[0])
                    except:
                        additive_id = i

                additives_list.append({
                    'id': additive_id or i,
                    'name_zh': name_zh,
                    'name_en': name_en,
                    'cns_code': cns_code,
                    'ins_code': ins_code,

```

```

        'function_category': function_category,
        'detail_url': detail_url
    })

    logger.info(f"成功获取 {len(additives_list)} 个添加剂列表")
    return additives_list

except Exception as e:
    logger.error(f"获取添加剂列表失败: {e}")
    return []

def scrape_additive_detail(self, additive_info: Dict) -> bool:
    """爬取单个添加剂的详情信息"""
    additive_id = additive_info['id']
    name_zh = additive_info['name_zh']
    detail_url = additive_info['detail_url']

    try:
        response = self.session.get(detail_url, timeout=15)
        response.encoding = 'utf-8'

        if response.status_code != 200:
            logger.warning(f"详情页面访问失败: {name_zh} - 状态码: {response.status_code}")
            return False

        soup = BeautifulSoup(response.text, 'html.parser')
        tables = soup.find_all('table')

        if len(tables) < 1:
            logger.warning(f"详情页面无表格数据: {name_zh}")
            return False

        # 解析基本信息
        basic_info = additive_info.copy()

        if tables[0]:
            basic_table = tables[0]
            basic_rows = basic_table.find_all('tr')

            for row in basic_rows:
                cells = row.find_all(['td', 'th'])
                if len(cells) >= 2:
                    field_name = cells[0].get_text(strip=True)

```

```

        field_value = cells[1].get_text(strip=True)

        if field_name == '质量规格标准':
            basic_info['quality_standard'] = field_value
        elif field_name == 'JECFA 规格资料':
            basic_info['jecfa_spec'] = field_value
        elif field_name == '备注':
            basic_info['remarks'] = field_value

# 确保字段完整
for field in ['quality_standard', 'jecfa_spec', 'remarks']:
    if field not in basic_info:
        basic_info[field] = ""

self.additives_basic.append(basic_info)

# 解析使用限量信息
usage_count = 0
for table_idx, table in enumerate(tables[1:], 1):
    usage_rows = table.find_all('tr')

    if len(usage_rows) < 2:
        continue

    header_row = usage_rows[0]
    headers = [th.get_text(strip=True) for th in header_row.find_all(['th', 'td'])]

    if not any(h in ['食品分类号', '食品名称', '最大使用量'] for h in headers):
        continue

    for row in usage_rows[1:]:
        cells = row.find_all(['td', 'th'])
        if len(cells) >= 3:
            category_code = cells[0].get_text(strip=True)
            food_name = cells[1].get_text(strip=True)
            max_usage = cells[2].get_text(strip=True)
            usage_remarks = cells[3].get_text(strip=True) if len(cells) > 3 else ""

            usage_info = {
                'additive_id': additive_id,
                'additive_name': name_zh,
                'category_code': category_code,
                'food_name': food_name,
                'max_usage': max_usage,

```



```

        detail_url = self.base_url + detail_url

        category_id = None
        if '/limit/' in detail_url:
            try:
                category_id =
            except:
                category_id = i

        categories_list.append({
            'id': category_id or i,
            'category_code': category_code,
            'category_name': category_name,
            'detail_url': detail_url
        })

    logger.info(f"成功获取 {len(categories_list)} 个食品分类列表")
    return categories_list

except Exception as e:
    logger.error(f"获取分类列表失败: {e}")
    return []

def scrape_category_detail(self, category_info: Dict) -> bool:
    """爬取单个分类的详情信息"""
    category_id = category_info['id']
    category_code = category_info['category_code']
    category_name = category_info['category_name']
    detail_url = category_info['detail_url']

    try:
        response = self.session.get(detail_url, timeout=15)
        response.encoding = 'utf-8'

        if response.status_code != 200:
            logger.warning(f"分类详情页面访问失败: {category_name} - 状态码: {response.status_code}")
            return False

        soup = BeautifulSoup(response.text, 'html.parser')
        tables = soup.find_all('table')

        if len(tables) < 1:

```

```

        logger.warning(f"分类详情页面无表格数据: {category_name}")
        return False

# 解析分类基本信息
basic_info = category_info.copy()

if tables and len(tables) >= 1:
    basic_table = tables[0]
    basic_rows = basic_table.find_all('tr')

    for row in basic_rows:
        cells = row.find_all(['td', 'th'])
        if len(cells) >= 2:
            field_name = cells[0].get_text(strip=True)
            field_value = cells[1].get_text(strip=True)

            if field_name in ['食品名称描述', '描述']:
                basic_info['description'] = field_value
            elif field_name in ['相关食品标准', '标准']:
                basic_info['standards'] = field_value

    for field in ['description', 'standards']:
        if field not in basic_info:
            basic_info[field] = ""

self.categories_basic.append(basic_info)

# 解析添加剂关联信息
additive_count = 0
for table_idx, table in enumerate(tables[1:], 1):
    additive_rows = table.find_all('tr')

    if len(additive_rows) < 2:
        continue

    header_row = additive_rows[0]
    headers = [th.get_text(strip=True) for th in header_row.find_all(['th', 'td'])]

    if not any(h in ['添加剂', 'CNS', 'INS', '最大使用量'] for h in headers):
        continue

    for row in additive_rows[1:]:
        cells = row.find_all(['td', 'th'])
        if len(cells) >= 3:

```

```

        additive_name = cells[0].get_text(strip=True)
        function = cells[1].get_text(strip=True) if len(cells) > 1 else ""
        max_usage = cells[2].get_text(strip=True) if len(cells) > 2 else ""
        cns_code = cells[3].get_text(strip=True) if len(cells) > 3 else ""
        ins_code = cells[4].get_text(strip=True) if len(cells) > 4 else ""
        remarks = cells[5].get_text(strip=True) if len(cells) > 5 else ""

        additive_usage = {
            'category_id': category_id,
            'category_code': category_code,
            'category_name': category_name,
            'additive_name': additive_name,
            'function': function,
            'max_usage': max_usage,
            'cns_code': cns_code,
            'ins_code': ins_code,
            'remarks': remarks
        }

        self.category_additives.append(additive_usage)
        additive_count += 1

    logger.info(f"成功爬取分类 {category_name}: 基本信息 + {additive_count} 个
添加剂")

    return True

except Exception as e:
    logger.error(f"爬取分类详情失败 {category_name}: {e}")
    return False

# ===== 其他数据爬取 =====

def scrape_other_data(self) -> bool:
    """爬取其他数据（加工助剂、酶制剂、香精香料）"""
    logger.info("开始爬取其他数据...")

    # 爬取加工助剂
    try:
        url = self.base_url + "/processing.html"
        response = self.session.get(url, timeout=15)
        response.encoding = 'utf-8'
        soup = BeautifulSoup(response.text, 'html.parser')

        tables = soup.find_all('table')

```



```

for table in tables:
    rows = table.find_all('tr')
    if len(rows) > 1:
        for i, row in enumerate(rows[1:], 1):
            cells = row.find_all(['td', 'th'])
            if len(cells) >= 1:
                row_data = [cell.get_text(strip=True) for cell in cells]

                aid_info = {
                    'name_zh': row_data[0] if row_data[0] else f"加工助剂_{i}",
                    'name_en': row_data[1] if len(row_data) > 1 else "",
                    'function': row_data[2] if len(row_data) > 2 else "",
                    'usage_scope': ' | '.join(row_data[3:]) if len(row_data) > 3
                    else "",
                    'cas_number': "",
                    'remarks': "",
                    'raw_data': ' | '.join(row_data)
                }

                self.processing_aids.append(aid_info)

        logger.info(f"成功爬取 {len(self.processing_aids)} 个加工助剂")
except Exception as e:
    logger.error(f"爬取加工助剂失败: {e}")

# 爬取酶制剂
try:
    url = self.base_url + "/enzyme.html"
    response = self.session.get(url, timeout=15)
    response.encoding = 'utf-8'
    soup = BeautifulSoup(response.text, 'html.parser')

    tables = soup.find_all('table')
    for table in tables:
        rows = table.find_all('tr')
        if len(rows) > 1:
            for i, row in enumerate(rows[1:], 1):
                cells = row.find_all(['td', 'th'])
                if len(cells) >= 1:
                    row_data = [cell.get_text(strip=True) for cell in cells]

                    enzyme_info = {
                        'name_zh': row_data[0] if row_data[0] else f"酶制剂_{i}",

```

```

        'name_en': row_data[1] if len(row_data) > 1 else "",
        'enzyme_source': row_data[2] if len(row_data) > 2 else
    "",
        'usage_scope': ' | '.join(row_data[3:]) if len(row_data) > 3
    else "",
        'ec_number': "",
        'cas_number': "",
        'remarks': "",
        'raw_data': ' | '.join(row_data)
    }

    self.enzymes.append(enzyme_info)

    logger.info(f"成功爬取 {len(self.enzymes)} 个酶制剂")
except Exception as e:
    logger.error(f"爬取酶制剂失败: {e}")

# 爬取香精香料（完整三个表格）
try:
    # 1. 爬取表 B.1 - 不得添加食品用香料、香精的食品名单
    url = self.base_url + "/spices.html"
    response = self.session.get(url, timeout=15)
    response.encoding = 'utf-8'
    soup = BeautifulSoup(response.text, 'html.parser')

    tables = soup.find_all('table')
    for table in tables:
        rows = table.find_all('tr')
        if len(rows) > 1:
            for i, row in enumerate(rows[1:], 1):
                cells = row.find_all(['td', 'th'])
                if len(cells) >= 2:
                    row_data = [cell.get_text(strip=True) for cell in cells]

                    spice_info = {
                        'category': 'B.1-禁用食品名单',
                        'name_zh': row_data[1] if len(row_data) > 1 else
row_data[0],
                        'name_en': "",
                        'type': '不得添加香精香料',
                        'code': row_data[0] if len(row_data) > 1 else "",
                        'fema_no': "",
                        'classification_table': 'B.1',
                        'remarks': '不得添加食品用香料、香精',

```

```

        'raw_data': ' | '.join(row_data)
    }

    self.spices.append(spice_info)

# 2. 爬取表 B.2 - 食品用天然香料
url_b2 = self.base_url + "/spices/type/b2.html"
response = self.session.get(url_b2, timeout=15)
response.encoding = 'utf-8'
soup = BeautifulSoup(response.text, 'html.parser')

tables = soup.find_all('table')
for table in tables:
    rows = table.find_all('tr')
    if len(rows) > 1:
        for i, row in enumerate(rows[1:], 1):
            cells = row.find_all(['td', 'th'])
            if len(cells) >= 3:
                row_data = [cell.get_text(strip=True) for cell in cells]

                spice_info = {
                    'category': row_data[0] if row_data[0] else 'B.2-天然香料',

                    'name_zh': row_data[1] if len(row_data) > 1 else '',
                    'name_en': row_data[2] if len(row_data) > 2 else '',
                    'type': '天然香料',
                    'code': row_data[3] if len(row_data) > 3 else '',
                    'fema_no': row_data[4] if len(row_data) > 4 else '',
                    'classification_table': row_data[5] if len(row_data) > 5

                else 'B.2',

                    'remarks': row_data[6] if len(row_data) > 6 else '',
                    'raw_data': ' | '.join(row_data)
                }

                self.spices.append(spice_info)

# 3. 爬取表 B.3 - 食品用合成香料
url_b3 = self.base_url + "/spices/type/b3.html"
response = self.session.get(url_b3, timeout=15)
response.encoding = 'utf-8'
soup = BeautifulSoup(response.text, 'html.parser')

tables = soup.find_all('table')
for table in tables:

```

```

        rows = table.find_all('tr')
        if len(rows) > 1:
            for i, row in enumerate(rows[1:], 1):
                cells = row.find_all(['td', 'th'])
                if len(cells) >= 3:
                    row_data = [cell.get_text(strip=True) for cell in cells]

                    spice_info = {
                        'category': row_data[0] if row_data[0] else 'B.3-合成香料',

                        'name_zh': row_data[1] if len(row_data) > 1 else '',
                        'name_en': row_data[2] if len(row_data) > 2 else '',
                        'type': '合成香料',
                        'code': row_data[3] if len(row_data) > 3 else '',
                        'fema_no': row_data[4] if len(row_data) > 4 else '',
                        'classification_table': row_data[5] if len(row_data) > 5
else 'B.3',

                        'remarks': row_data[6] if len(row_data) > 6 else '',
                        'raw_data': ' | '.join(row_data)
                    }

                self.spices.append(spice_info)

        logger.info(f"成功爬取香精香料: B.1 表+B.2 表(天然)+B.3 表(合成) =
{len(self.spices)} 个")
        except Exception as e:
            logger.error(f"爬取香精香料失败: {e}")

        return True

# ===== 主要爬取流程 =====

def scrape_all_data(self) -> bool:
    """爬取所有数据"""
    logger.info("开始完整数据爬取...")

    print("📄 GB 2760-2024 最终完整数据爬虫")
    print("=" * 60)
    print("100% 真实数据爬取，建立完整多对多关系")
    print()

    # 1. 爬取添加剂详情
    print("📄 第一阶段：爬取添加剂详情数据")

```

```
additives_list = self.get_additive_list()
if not additives_list:
    logger.error("无法获取添加剂列表")
    return False

success_count = 0
for i, additive_info in enumerate(additives_list, 1):
    if self.scrape_additive_detail(additive_info):
        success_count += 1

    if i % 10 == 0:
        logger.info(f"添加剂进度: {i}/{len(additives_list)} 完成, 成功 {success_count}
个")
```

```
        time.sleep(1)

print(f"☑ 添加剂数据爬取完成: {success_count}/{len(additives_list)}")
```

2. 爬取分类详情

```
print("\n📌 第二阶段: 爬取食品分类详情数据")
categories_list = self.get_category_list()
if not categories_list:
    logger.error("无法获取分类列表")
    return False

success_count = 0
for i, category_info in enumerate(categories_list, 1):
    if self.scrape_category_detail(category_info):
        success_count += 1

    if i % 10 == 0:
        logger.info(f"分类进度: {i}/{len(categories_list)} 完成, 成功 {success_count}
个")
```

```
        time.sleep(1)

print(f"☑ 分类数据爬取完成: {success_count}/{len(categories_list)}")
```

3. 爬取其他数据

```
print("\n📌 第三阶段: 爬取其他数据")
self.scrape_other_data()
print("☑ 其他数据爬取完成")
```

```
return True
```

```

def save_to_excel(self, filename: str = 'final_gb2760_data.xlsx') -> bool:
    """保存完整数据到 Excel"""
    try:
        filepath = __import__('pathlib').Path(__file__).parent / filename
        #filepath = f"d:/html+css/GB 2760-2024/{filename}"

        with pd.ExcelWriter(filepath, engine='openpyxl') as writer:

            # 添加剂基本信息表
            if self.additives_basic:
                basic_df = pd.DataFrame(self.additives_basic)
                basic_df.to_excel(writer, sheet_name='添加剂基本信息', index=False)

            # 添加剂使用限量表
            if self.additives_usage:
                usage_df = pd.DataFrame(self.additives_usage)
                usage_df.to_excel(writer, sheet_name='添加剂使用限量', index=False)

            # 分类基本信息表
            if self.categories_basic:
                cat_basic_df = pd.DataFrame(self.categories_basic)
                cat_basic_df.to_excel(writer, sheet_name='分类基本信息', index=False)

            # 分类-添加剂关联表
            if self.category_additives:
                cat_add_df = pd.DataFrame(self.category_additives)
                cat_add_df.to_excel(writer, sheet_name='分类 - 添加剂关联',
index=False)

            # 加工助剂表
            if self.processing_aids:
                aids_df = pd.DataFrame(self.processing_aids)
                aids_df.to_excel(writer, sheet_name='加工助剂', index=False)

            # 酶制剂表
            if self.enzymes:
                enzymes_df = pd.DataFrame(self.enzymes)
                enzymes_df.to_excel(writer, sheet_name='酶制剂', index=False)

            # 香精香料表
            if self.spices:
                spices_df = pd.DataFrame(self.spices)
                spices_df.to_excel(writer, sheet_name='香精香料', index=False)

```

```

        # 数据统计表
        stats_data = [
            {'数据类型': '添加剂基本信息', '数量': len(self.additives_basic), '说明': '包含完整的添加剂基本信息'},
            {'数据类型': '添加剂使用限量', '数量': len(self.additives_usage), '说明': '添加剂在不同食品分类中的使用限量'},
            {'数据类型': '分类基本信息', '数量': len(self.categories_basic), '说明': '包含完整的分类基本信息'},
            {'数据类型': '分类-添加剂关联', '数量': len(self.category_additives), '说明': '分类可以使用的添加剂列表'},
            {'数据类型': '加工助剂', '数量': len(self.processing_aids), '说明': '加工助剂基本信息'},
            {'数据类型': '酶制剂', '数量': len(self.enzymes), '说明': '酶制剂基本信息'},
            {'数据类型': '香精香料', '数量': len(self.spices), '说明': '香精香料基本信息'}
        ]
        stats_df = pd.DataFrame(stats_data)
        stats_df.to_excel(writer, sheet_name='数据统计', index=False)

    logger.info(f"完整数据已保存到: {filepath}")
    return True

except Exception as e:
    logger.error(f"保存 Excel 文件失败: {e}")
    return False

def main():
    """主函数"""
    scraper = FinalGB2760Scraper()

    if scraper.scrape_all_data():
        if scraper.save_to_excel():
            print("\n🎉 完整数据爬取和保存成功！")
            print("=" * 50)
            print(f"📊 添加剂基本信息: {len(scraper.additives_basic)} 个")
            print(f"📊 添加剂使用限量: {len(scraper.additives_usage)} 条")
            print(f"📊 分类基本信息: {len(scraper.categories_basic)} 个")
            print(f"📊 分类-添加剂关联: {len(scraper.category_additives)} 条")
            print(f"📊 加工助剂: {len(scraper.processing_aids)} 个")
            print(f"📊 酶制剂: {len(scraper.enzymes)} 个")
            print(f"📊 香精香料: {len(scraper.spices)} 个")
            print("\n📁 文件保存位置:")

```

```

        print(__import__('pathlib').Path(__file__).parent / "final_gb2760_data.xlsx")
        print(__import__('pathlib').Path(__file__).parent / "final_scraper.log")
    else:
        print("✘ 数据保存失败！")
    else:
        print("✘ 数据爬取失败！")

if __name__ == "__main__":
    main()

```

```

validate_sql_match.py
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""

```

```

SQL 脚本与数据文件匹配性验证工具
"""

```

```

import pandas as pd
import re
import logging
from typing import Dict, List, Set

```

```

# 配置日志
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

```

```

class SQLDataMatcher:
    """SQL 脚本与数据文件匹配性验证器"""

    def __init__(self):
        self.excel_file = 'final_gb2760_data.xlsx'
        self.sql_file = 'database_schema.sql'

        # 预期的表与工作表映射
        self.table_sheet_mapping = {
            'additive': '添加剂基本信息',
            'additive_usage': '添加剂使用限量',
            'food_category': '分类基本信息',
            'category_additive': '分类-添加剂关联',
            'processing_aid': '加工助剂',
            'enzyme': '酶制剂',
            'flavor': '香精香料'
        }
    }

```



```

def load_excel_structure(self) -> Dict[str, List[str]]:
    """加载 Excel 文件的结构信息"""
    structure = {}

    try:
        with pd.ExcelFile(self.excel_file) as xls:
            for sheet_name in xls.sheet_names:
                if sheet_name != '数据统计': # 跳过统计表
                    df = pd.read_excel(self.excel_file, sheet_name=sheet_name)
                    structure[sheet_name] = list(df.columns)

        logger.info(f"成功加载 Excel 结构，包含 {len(structure)} 个工作表")
        return structure

    except Exception as e:
        logger.error(f"加载 Excel 文件失败: {e}")
        return {}

def parse_sql_structure(self) -> Dict[str, List[str]]:
    """解析 SQL 文件中的表结构"""
    structure = {}

    try:
        with open(self.sql_file, 'r', encoding='utf-8') as f:
            sql_content = f.read()

        # 简化的表结构解析
        lines = sql_content.split('\n')
        current_table = None
        current_columns = []
        in_table_def = False

        for line in lines:
            line = line.strip()

            # 检测 CREATE TABLE 语句
            if line.upper().startswith('CREATE TABLE'):
                # 保存上一个表
                if current_table and current_columns:
                    structure[current_table] = current_columns.copy()

                # 提取表名
                table_match = re.search(r'CREATE TABLE\s+(\w+)\s*', line,
re.IGNORECASE)

```

```

        if table_match:
            current_table = table_match.group(1)
            current_columns = []
            in_table_def = True
            continue

    if in_table_def:
        # 检测表定义结束
        if line.startswith(';') or line == '':
            if current_table and current_columns:
                structure[current_table] = current_columns.copy()
            in_table_def = False
            current_table = None
            current_columns = []
            continue

        # 跳过注释和空行
        if line.startswith('--') or not line or line.startswith('/'):
            continue

        # 跳过索引和约束
        skip_keywords = ['INDEX', 'KEY', 'CONSTRAINT', 'FOREIGN', 'PRIMARY',
                        'KEY',
                        'UNIQUE KEY', 'FULLTEXT', 'ENGINE', 'COMMENT=',
                        'AUTO_INCREMENT']

        if any(keyword in line.upper() for keyword in skip_keywords):
            continue

        # 提取字段名
        field_match = re.match(r'\s*(\w+)\s+', line)
        if field_match:
            field_name = field_match.group(1)
            # 过滤保留字
            if field_name.upper() not in ['PRIMARY', 'UNIQUE', 'INDEX', 'KEY',
            'CONSTRAINT']:
                current_columns.append(field_name)

    # 处理最后一个表
    if current_table and current_columns:
        structure[current_table] = current_columns

    logger.info(f"成功解析 SQL 结构，包含 {len(structure)} 个表")
    for table, columns in structure.items():
        logger.info(f"    {table}: {len(columns)} 个字段 - {columns[:3]}{'...' if

```

```
len(columns) > 3 else "")
```

```
    return structure
```

```
except Exception as e:
```

```
    logger.error(f"解析 SQL 文件失败: {e}")
```

```
    return {}
```

```
def validate_matching(self) -> bool:
```

```
    """验证 SQL 表结构与 Excel 数据结构的匹配性"""
```

```
    print("\n SQL 脚本与数据文件匹配性验证")
```

```
    print("=" * 60)
```

```
    # 加载结构
```

```
    excel_structure = self.load_excel_structure()
```

```
    sql_structure = self.parse_sql_structure()
```

```
    if not excel_structure or not sql_structure:
```

```
        print("✗ 无法加载数据结构")
```

```
        return False
```

```
    overall_match = True
```

```
    # 验证每个表的匹配性
```

```
    for table_name, sheet_name in self.table_sheet_mapping.items():
```

```
        print(f"\n 验证表: {table_name} ↔ 工作表: {sheet_name}")
```

```
        if table_name not in sql_structure:
```

```
            print(f"✗ SQL 中缺少表: {table_name}")
```

```
            overall_match = False
```

```
            continue
```

```
        if sheet_name not in excel_structure:
```

```
            print(f"✗ Excel 中缺少工作表: {sheet_name}")
```

```
            overall_match = False
```

```
            continue
```

```
        sql_columns = set(sql_structure[table_name])
```

```
        excel_columns = set(excel_structure[sheet_name])
```

```
    # 检查字段匹配
```

```
    missing_in_sql = excel_columns - sql_columns
```

```
    missing_in_excel = sql_columns - excel_columns
```

```
    common_fields = sql_columns & excel_columns
```

```

        if missing_in_sql:
            print(f"❌ SQL 表中缺少字段: {' '.join(missing_in_sql)}")
            overall_match = False

        if missing_in_excel:
            print(f"❌ Excel 中缺少字段: {' '.join(missing_in_excel)}")
            overall_match = False

    match_rate = len(common_fields) / max(len(sql_columns), len(excel_columns)) *

100

    if match_rate >= 95:
        print(f"✅ 字段匹配率 : {match_rate:.1f}%
({len(common_fields)}/{max(len(sql_columns), len(excel_columns))})")
    elif match_rate >= 80:
        print(f"🟡 字段匹配率 : {match_rate:.1f}%
({len(common_fields)}/{max(len(sql_columns), len(excel_columns))})")
    else:
        print(f"❌ 字段匹配率 : {match_rate:.1f}%
({len(common_fields)}/{max(len(sql_columns), len(excel_columns))})")
        overall_match = False

# 输出总结
print("\n" + "=" * 60)
if overall_match:
    print("✅ 验证结果: SQL 脚本与数据文件结构完全匹配!")
    print("✅ 可以直接使用 data_import.py 进行数据导入")
else:
    print("❌ 验证结果: 发现结构不匹配问题")
    print("❌ 需要调整 SQL 脚本或数据文件结构")

print("=" * 60)
return overall_match

def generate_import_summary(self):
    """生成导入准备情况总结"""
    excel_structure = self.load_excel_structure()

    print("\n📊 数据导入准备情况:")
    print("-" * 40)

    total_records = 0
    for sheet_name in excel_structure:

```

```

        if sheet_name != '数据统计':
            df = pd.read_excel(self.excel_file, sheet_name=sheet_name)
            record_count = len(df)
            total_records += record_count

        # 匹配表名
        table_name = None
        for t_name, s_name in self.table_sheet_mapping.items():
            if s_name == sheet_name:
                table_name = t_name
                break

        print(f"    {sheet_name} → {table_name}: {record_count:,} 条记录")

    print(f"\n    总记录数: {total_records:,} 条")
    print(f"    数据文件: final_gb2760_data.xlsx")
    print(f"    目标数据库: gb2760_db")
    print(f"    导入工具: data_import.py")

def main():
    """主函数"""
    print("    GB 2760 数据库导入准备验证工具")
    print("=" * 60)

    matcher = SQLDataMatcher()

    # 验证匹配性
    is_matched = matcher.validate_matching()

    # 生成导入总结
    matcher.generate_import_summary()

    if is_matched:
        print("\n ☒ 准备就绪，可以开始数据导入:")
        print("    python data_import.py")
    else:
        print("\n ☐ 需要先解决结构匹配问题")

    return is_matched

if __name__ == "__main__":
    main()

```